Design YouTube / Streaming Service

Functional Requirements :
1. User (Creator) can upload a video.
2. Users can view / stream a video.

Non Functional Req:
1. Prefer A over C in CAP.

Not Covered :
1. Likes/disliked
2. Channel management
3. comments.

Q : CAP ? Consistency vs Availibility.
A : Availibility

Q: any restriction on video size ?
A : No restriction. File can be in GBS.

Q : DAU ?
A : 100 Million DAU.
-> 1 million views are uploaded / day
-> 100 million videos are watched / day.
on Avg : 1 user -> 1 video watch/day

Q : Are there compressions internally ?
A : yes.

Back of the envelope calculations:

OPS, Storage Unit.

OPS :
DAU : 100 million
Read OPS : 100 million videos watch / day
Write OPS : 1 Million videos uploaded / day.

Storage unit : (for 5 years)
Avg video size (After compression) : 1 GB.

Power of 2 : $2^{10} = 10^3$

Basic Flow :
1. Posting a video.

POST :

SQL / NO-SQl

SQL -> BLOB (Binary Large OBject)
Videos
Images

Q : Should we Upload in DB ?
A : No

Reasons :
-> DBs are expensive and its memory will become full.
-> DB will become slow.
-> Query not possible on Video
-> Migration (SQl -> NOSQl)

Video metadata ?
VideoId,
VideoName,
Desc
...

Sample Metadata:
{
videoId : 123,
videoName : "something",
videoDesc : "Anything",
timestamps : [
]
}

NoSQL -> MongoDb, Casandra

API End points :
user can upload a video
user can watch a video.

1. Uploading a video:
POST /upload
{
video,
videoMetaData
}

Response : 200 ok

2. Watch / Stream a video:
GET
Response :
{
Video,
VideoMetaData
}

200 ok

Amazon S3:

Problems :
1. Server will become extremely busy.
2. Even if we introduce MQ & Workers MQ -> limited storage.
This idea will not work.

Client should be responsible to upload video not server.

Problems :
1. No compression is used.
2. No video chunks

Concepts :
1. Video Codec
Codec : Encoder-Decoder
-> Responsible to compress and decompress a video.
-> Reduce size
-> Try that Quality will remain same.
-> Lossless compression
-> Can also do lossy compression.

Trade-OFF

High compression -> High chance for lossy compression

Ex : MPEG-2, MPEG-4, h.265

2. Bitrate
No. of bits transmitted at a given time.
KBPS, MBPS

Better Approach: (POST)

For Streaming

Problem :
Slow internet connection can lead to too much buffering.

Flexible System
Client -> 240P, 480P, 720P, 1080P, ....

Automatic Format Selection :

Adaptive Bitrate Streaming

Client -> bitrate (SFTP)

What is the client's downloading speed ?

< 100 MBPS = -> 1080P
50MBPS -> 720P

Final Design of posting a video




Streaming a Video